

1.Traversing Relationships

Our domain model shows that the *ACTED_IN* relationship goes in an outgoing direction from the *Person* node so we can add the direction in our pattern. We often refer to this as a **traversal**.

MATCH (p:Person {name: 'Tom Hanks'})-[:ACTED_IN]->()

2.Returning Related Nodes

Our data model dictates that the node at the other end of that relationship will be Movie node, so we don’t necessarily need to specify the :Movie label in the node - instead we will use the variable m.

*MATCH (p:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(m)
RETURN m.title*

3. Specifying Node Labels

That is, if we had multiple types of nodes at the end of the ACTED_IN relationships in our graph, we could make sure that we only return movies.

*MATCH (p:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(m:Movie)
RETURN m.title*

Challenges

1.There is DIRECTED relationship between the Person nodes and Movie nodes - it represents the people who directed the movies

MATCH (p: Person)-[:DIRECTED]->(m:Movie) RETURN p.name

2.who directed movie Cloud Atlas

MATCH (p:Person)-[:DIRECTED]->(m: Movie {title: 'Cloud Atlas'})
RETURN p

3.which movie Emil Eifrem acted in

MATCH (p:Person {name: "Emil Eifrem"})-[:ACTED_IN]->(m: Movie)
RETURN m;